# RW Table (Read-Write Table)

## 목적

RW Table은 ZKEVM에서 가장 중요한 테이블 중 하나로, EVM 실행 중 발생하는 **모든 읽기/쓰기 연산**을 추적하고 검증합니다. 이 테이블은 EVM Circuit과 State Circuit 간의 핵심 인터페이스 역할을 하며, 실행 추적(execution trace)과 상태 변경(state changes)을 연결합니다.

## 테이블 스키마

**파일**: `zkevm-circuits/src/table.rs` (line 617)

```
pub struct RwTable {
/// 행 활성화 여부
pub q_enable: Column<Fixed>,
/// Read-Write 카운터 (연산 순서)
pub rw_counter: Column<Advice>,
/// 쓰기 연산 여부 (true: Write, false: Read)
pub is_write: Column<Advice>,
/// 연산 타입 태그 (RwTableTag)
pub tag: Column<Advice>,
/// Key1: ID (Call ID, Tx ID 등)
pub id: Column<Advice>,
/// Key2: 주소 (메모리 주소, 스택 포인터, 계정 주소 등)
```

```rust
pub  address: Column<Advice>,

/// Key3: 필드 태그 (AccountFieldTag, CallContextFieldTag 등)

pub  field_tag: Column<Advice>,

/// Key3: 스토리지 키 (AccountStorage용)

pub  storage_key: Column<Advice>,

/// 값 (현재 값)

pub  value: Column<Advice>,

/// 이전 값 (쓰기 전 값)

pub  value_prev: Column<Advice>,

/// 보조 데이터 1

pub  aux1: Column<Advice>,

/// 보조 데이터 2 (Committed Value)

pub  aux2: Column<Advice>,

}
```

## 컬럼 설명

### 1. q_enable (Fixed)

- 해당 행이 유효한 RW 연산인지 표시
- 패딩 행은 `q_enable = 0`

### 2. rw_counter (Advice)

- 모든 RW 연산에 대한 **전역 카운터**
- 연산이 발생한 순서를 나타냄

- EVM 실행 순서와 일치하며, State Circuit에서 정렬 키로 사용

**3. is_write (Advice)**

- `1` : 쓰기 연산 (Write)
- `0` : 읽기 연산 (Read)

**4. tag (Advice)**

- 연산 타입을 식별하는 태그 ( `RwTableTag` )
- 각 태그에 따라 다른 컬럼들의 의미가 달라짐

**5. id (Advice)**

- 연산의 컨텍스트를 식별하는 ID
- **Stack/Memory**: Call ID
- **TxAccessList/TxRefund**: Transaction ID
- **Account/AccountStorage**: 사용되지 않음 (0)
- **CallContext**: Call ID
- **TxLog/TxReceipt**: Transaction ID

**6. address (Advice)**

- 연산 대상의 주소
- **Stack**: 스택 포인터 (1024 - stack_pointer)
- **Memory**: 메모리 주소
- **Account/AccountStorage**: 계정 주소 (20바이트)
- **CallContext**: 사용되지 않음
- **TxLog**: 로그 주소 (tx_id, log_id, field_tag, index 조합)

**7. field_tag (Advice)**

- 세부 필드를 식별하는 태그
- **Account**: `AccountFieldTag` (Nonce, Balance, CodeHash 등)
- **CallContext**: `CallContextFieldTag` (CallerId, Depth, Value 등)
- **TxLog**: `TxLogFieldTag` (Address, Topic, Data)
- **TxReceipt**: `TxReceiptFieldTag` (PostStateOrStatus, CumulativeGasUsed, LogLength)

**8. storage_key (Advice, SecondPhase)**

- **AccountStorage**에서 스토리지 슬롯 키 (32바이트)
- 다른 연산 타입에서는 사용되지 않음
- RLC로 인코딩되어 SecondPhase에 할당

**9. value (Advice, SecondPhase)**

- 연산의 현재 값
- **Read**: 읽은 값
- **Write**: 쓴 값

**10. value_prev (Advice, SecondPhase)**

- 쓰기 연산 전의 이전 값
- **Read**: `value` 와 동일
- **Write**: 쓰기 전 값

**11. aux1 (Advice, SecondPhase)**

- 보조 데이터 1
- **TxLog**: `topic_count` (로그의 토픽 개수)

- **AccountStorage**: 사용되지 않음

**12. aux2 (Advice, SecondPhase)**

- 보조 데이터 2 (Committed Value)
- **AccountStorage**: 트랜잭션 시작 시 커밋된 값
- 다른 연산에서는 사용되지 않음

## RwTableTag 열거형

**파일**: `zkevm-circuits/src/table.rs` (line 467)

```
pub  enum RwTableTag {
Start = 1, // 패딩용 시작 태그
Stack, // 스택 연산
Memory, // 메모리 연산
TxAccessListAccount, // Tx Access List 계정
TxAccessListAccountStorage, // Tx Access List 스토리지
TxRefund, // Tx 환불
Account, // 계정 필드
AccountStorage, // 계정 스토리지
AccountTransientStorage, // 계정 임시 스토리지 (EIP-1153)
CallContext, // 호출 컨텍스트
TxLog, // Tx 로그
TxReceipt, // Tx 영수증
```

```
}
```

## Reversible 연산

일부 RW 연산은 **되돌릴 수 있는(reversible)** 연산입니다. 이는 트랜잭션이 revert될 때 상태가 원래대로 복구되어야 함을 의미합니다.

```
impl RwTableTag {
pub  fn  is_reversible(self) -> bool {
matches!(
self,
RwTableTag::TxAccessListAccount
| RwTableTag::TxAccessListAccountStorage
| RwTableTag::TxRefund
| RwTableTag::Account
| RwTableTag::AccountStorage
| RwTableTag::AccountTransientStorage
)
}
}
```

**Reversible 연산:**

- `TxAccessListAccount`
- `TxAccessListAccountStorage`
- `TxRefund`

- `Account`
- `AccountStorage`
- `AccountTransientStorage`

**Non-reversible 연산:**

- `Stack`
- `Memory`
- `CallContext`
- `TxLog`
- `TxReceipt`

# RW 타입별 데이터 구조

각 `RwTableTag` 에 따라 컬럼들이 다르게 사용됩니다.

### 1. Stack

스택 push/pop 연산을 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::Stack` |

| `id` | Call ID |

| `address` | 1024 - stack_pointer |

| `field_tag` | 0 |

| `storage_key` | 0 |

| `value` | 스택 값 (32바이트 word) |

| `value_prev` | Read: `value` 와 동일
Write: 이전 값 |

| `aux1` | 0 |

| `aux2` | 0 |

**예시**: `PUSH1 0x42` 후 `POP`

```
 rw_counter | is_write | tag | id | address | value | value_prev

 -----------|----------|-------|----|---------| ------|------------

 100 | 1 | Stack | 1 | 1023 | 0x42 | 0x00

 101 | 0 | Stack | 1 | 1023 | 0x42 | 0x42
```

## 2. Memory

메모리 읽기/쓰기 연산을 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::Memory` |

| `id` | Call ID |

| `address` | 메모리 주소 (바이트 단위) |

| `field_tag` | 0 |

| `storage_key` | 0 |

| `value` | 메모리 값 (1바이트) |

| `value_prev` | Read: `value` 와 동일
Write: 이전 값 |

| `aux1` | 0 |

| aux2 | 0 |

**특징**:

- 메모리는 **바이트 단위**로 추적됨
- 32바이트 word 연산은 32개의 RW 행으로 분해됨

**예시**: `MSTORE(0x00, 0x1234)`

```
rw_counter | is_write | tag | id | address | value | value_prev

-----------|----------|--------|----|---------| ------|------------

200 | 1 | Memory | 1 | 30 | 0x12 | 0x00

201 | 1 | Memory | 1 | 31 | 0x34 | 0x00
```

### 3. TxAccessListAccount

EIP-2930 Access List의 계정 접근을 추적합니다.

| 컬럼 | 값 |

|------|-----|

| tag | RwTableTag::TxAccessListAccount |

| id | Transaction ID |

| address | 계정 주소 |

| field_tag | 0 |

| storage_key | 0 |

| value | is_warm (1: warm, 0: cold) |

| value_prev | 이전 is_warm 값 |

| aux1 | 0 |

| `aux2` | 0 |

## 4. TxAccessListAccountStorage

EIP-2930 Access List의 스토리지 접근을 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::TxAccessListAccountStorage` |

| `id` | Transaction ID |

| `address` | 계정 주소 |

| `field_tag` | 0 |

| `storage_key` | 스토리지 키 (RLC) |

| `value` | `is_warm` (1: warm, 0: cold) |

| `value_prev` | 이전 `is_warm` 값 |

| `aux1` | 0 |

| `aux2` | 0 |

## 5. TxRefund

가스 환불을 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::TxRefund` |

| `id` | Transaction ID |

| `address` | 0 |

| `field_tag` | 0 |

| storage_key | 0 |

| value | 현재 환불 가스량 |

| value_prev | 이전 환불 가스량 |

| aux1 | 0 |

| aux2 | 0 |

## 6. Account

계정 필드(Nonce, Balance, CodeHash 등)를 추적합니다.

| 컬럼 | 값 |

|------|-----|

| tag | RwTableTag::Account |

| id | 0 |

| address | 계정 주소 |

| field_tag | AccountFieldTag |

| storage_key | 0 |

| value | 필드 값 |

| value_prev | 이전 필드 값 |

| aux1 | 0 |

| aux2 | 0 |

**AccountFieldTag**:

```
pub  enum AccountFieldTag {
Nonce, // 계정 nonce
Balance, // 계정 잔액
```

```
KeccakCodeHash, // 코드의 Keccak 해시

CodeHash, // 코드의 Poseidon 해시

CodeSize, // 코드 크기

NonExisting, // 계정 존재 여부

}
```

**예시**: 계정 잔액 변경

```
rw_counter | is_write | tag | address | field_tag | value | value_prev

-----------|----------|---------|---------|-----------|-------|------------

300 | 0 | Account | 0x1234 | Balance | 100 | 100

301 | 1 | Account | 0x1234 | Balance | 90 | 100
```

## 7. AccountStorage

계정 스토리지 슬롯을 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::AccountStorage` |

| `id` | 0 |

| `address` | 계정 주소 |

| `field_tag` | 0 |

| `storage_key` | 스토리지 키 (RLC) |

| `value` | 스토리지 값 |

| `value_prev` | 이전 스토리지 값 |

| `aux1` | 0 |

| `aux2` | Committed value (트랜잭션 시작 시 값) |

**특징**:

- `aux2` 는 EIP-2200 가스 계산을 위한 committed value 저장
- `storage_key` 는 RLC로 인코딩됨

**예시**: `SSTORE(key, value)`

```
rw_counter | is_write | tag | address | storage_key | value | value_prev | aux2
-----------|----------|----------------|---------|-------------|-------|--------
400 | 0 | AccountStorage | 0x1234 | key_rlc | 10 | 10 | 10
401 | 1 | AccountStorage | 0x1234 | key_rlc | 20 | 10 | 10
```

## 8. AccountTransientStorage

EIP-1153 임시 스토리지를 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::AccountTransientStorage` |

| `id` | 0 |

| `address` | 계정 주소 |

| `field_tag` | 0 |

| `storage_key` | 스토리지 키 (RLC) |

| `value` | 임시 스토리지 값 |

| `value_prev` | 이전 임시 스토리지 값 |

| `aux1` | 0 |

| `aux2` | 0 |

**특징**:

- 트랜잭션 종료 시 자동으로 초기화됨
- `TLOAD`, `TSTORE` opcode에서 사용

## 9. CallContext

호출 컨텍스트 필드를 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::CallContext` |

| `id` | Call ID |

| `address` | 0 |

| `field_tag` | `CallContextFieldTag` |

| `storage_key` | 0 |

| `value` | 필드 값 |

| `value_prev` | Read: `value` 와 동일
Write: 이전 값 |

| `aux1` | 0 |

| `aux2` | 0 |

**CallContextFieldTag** (일부):

```
pub enum CallContextFieldTag {
```

```
RwCounterEndOfReversion, // Revert 시 복구할 rw_counter

CallerId, // 호출자 ID

TxId, // 트랜잭션 ID

Depth, // 호출 깊이

CallerAddress, // 호출자 주소

CalleeAddress, // 피호출자 주소

CallDataOffset, // CallData 오프셋

CallDataLength, // CallData 길이

Value, // 전송 값

IsSuccess, // 호출 성공 여부

IsPersistent, // 상태 지속 여부

IsStatic, // Static 호출 여부

GasLeft, // 남은 가스

// ... 더 많은 필드

}
```

## 10. TxLog

트랜잭션 로그(이벤트)를 추적합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `RwTableTag::TxLog` |

| `id` | Transaction ID |

| `address` | 로그 주소 (tx_id, log_id, field_tag, index 조합) |

| field_tag | TxLogFieldTag |

| storage_key | 0 |

| value | 로그 데이터 |

| value_prev | value 와 동일 |

| aux1 | topic_count (토픽 개수) |

| aux2 | 0 |

**TxLogFieldTag**:

```
pub  enum TxLogFieldTag {
Address = 1, // 로그를 발생시킨 컨트랙트 주소
Topic, // 로그 토픽
Data, // 로그 데이터
}
```

## 11. TxReceipt

트랜잭션 영수증 정보를 추적합니다.

| 컬럼 | 값 |

|------|-----|

| tag | RwTableTag::TxReceipt |

| id | Transaction ID |

| address | 0 |

| field_tag | TxReceiptFieldTag |

| storage_key | 0 |

| value | 필드 값 |

| value_prev | value 와 동일 |

| aux1 | 0 |

| aux2 | 0 |

**TxReceiptFieldTag**:

```
pub  enum TxReceiptFieldTag {
PostStateOrStatus = 1, // Tx 결과 (1: success, 0: fail)
CumulativeGasUsed, // 누적 가스 사용량
LogLength, // 로그 개수
}
```

## State Circuit과의 연동

RW Table은 **EVM Circuit**과 **State Circuit** 간의 핵심 인터페이스입니다.

### 데이터 흐름

```
┌──────────────┐  ┌──────────────┐
| EVM Circuit | |State Circuit |
| | | |
| - Opcode | | - RW 정렬 |
| 실행 | | - 제약 조건 |
| - RW 연산 | | 검증 |
```

```
| 생성 | | - MPT 증명 |
└──────┬──────┘ └──────▲──────┘
| |
| Lookup | Write
| (검증) | (할당)
| |
└──────────► RW Table ◄──────────┘
(공유 테이블)
```

## EVM Circuit의 역할

1. **Lookup 수행**: EVM Circuit은 RW Table을 조회하여 읽기/쓰기 연산을 검증
2. **rw_counter 추적**: 각 opcode 실행 시 rw_counter를 증가시킴
3. **제약 조건**: Lookup을 통해 실제 상태 변경이 RW Table에 기록되었는지 확인

**예시 (EVM Circuit에서 SLOAD)**:

```
// EVM Circuit에서 SLOAD 실행
let  storage_value = meta.lookup_any("storage read", |meta| {
vec![
1.expr(), // q_enable
rw_counter.expr(), // rw_counter
0.expr(), // is_write = 0 (Read)
RwTableTag::AccountStorage.expr(), // tag
0.expr(), // id
```

```
callee_address.expr(), // address

0.expr(), // field_tag

storage_key_rlc.expr(), // storage_key

storage_value.expr(), // value

storage_value.expr(), // value_prev (Read이므로 동일)

0.expr(), // aux1

committed_value.expr(), // aux2

]

});
```

### State Circuit의 역할

1. **RW Table 할당**: State Circuit은 RW Table에 실제 데이터를 할당
2. **정렬 및 검증**: RW 연산을 정렬하고 상태 일관성을 검증
3. **MPT 증명**: 상태 변경에 대한 Merkle Patricia Trie 증명 생성

**정렬 키 (Lexicographic Ordering)**:

```
(tag, id, address, field_tag, storage_key, rw_counter)
```

State Circuit은 이 순서로 RW 연산을 정렬하여:

- 같은 위치에 대한 연산들을 그룹화
- 읽기/쓰기 순서 검증
- 상태 전이 검증

## RW 연산 생성 과정

RW 연산은 **Bus-Mapping** 단계에서 생성됩니다.

## 1. Geth 트레이스 수집

```
EVM 실행 → Geth Tracer → Execution Trace
```

## 2. Bus-Mapping 변환

**파일**: `bus-mapping/src/operation.rs`

```rust
pub enum RwOp {
Stack { ... },
Memory { ... },
AccountStorage { ... },
// ... 기타 연산
}
```

각 opcode 실행 시 해당하는 `RwOp` 가 생성됩니다.

## 3. RwMap 구축

**파일**: `zkevm-circuits/src/witness/rw.rs`

```rust
pub struct RwMap {
pub rws: Vec<Rw>,
}
```

```
impl RwMap {
pub fn table_assignments_prepad(
rws: &[Rw],
n_rows: usize,
) -> (Vec<Rw>, usize) {
// RW 연산을 정렬하고 패딩 추가
}
}
```

## 4. RW Table 할당

```
impl RwTable {
pub fn load<F: Field>(
&self,
layouter: &mut impl Layouter<F>,
rws: &[Rw],
n_rows: usize,
challenges: Value<F>,
) -> Result<(), Error> {
// RW 연산을 테이블에 할당
}
}
```

## 예시: CALL opcode의 RW 연산

`CALL` opcode는 여러 RW 연산을 생성합니다:

```
// 1. CallContext 읽기 (현재 컨텍스트 정보)
RW { tag: CallContext, id: caller_id, field_tag: GasLeft, is_write: false, ... }
RW { tag: CallContext, id: caller_id, field_tag: CallerAddress, is_write: false,

// 2. Account 읽기 (피호출자 계정 정보)
RW { tag: Account, address: callee, field_tag: Balance, is_write: false, ... }
RW { tag: Account, address: callee, field_tag: CodeHash, is_write: false, ... }

// 3. Account 쓰기 (잔액 전송)
RW { tag: Account, address: caller, field_tag: Balance, is_write: true, value: n
RW { tag: Account, address: callee, field_tag: Balance, is_write: true, value: n

// 4. CallContext 쓰기 (새 컨텍스트 생성)
RW { tag: CallContext, id: callee_id, field_tag: CallerId, is_write: true, value
RW { tag: CallContext, id: callee_id, field_tag: Depth, is_write: true, value: d
// ... 더 많은 CallContext 필드
```

## 성능 최적화

### 1. SecondPhase 컬럼

RLC를 사용하는 컬럼들은 SecondPhase에 할당됩니다:

- `storage_key` : 스토리지 키 RLC
- `value` , `value_prev` : 값 (RLC 가능)
- `aux1` , `aux2` : 보조 데이터

### 2. 정렬 최적화

State Circuit은 RW 연산을 정렬하여:

- 같은 위치에 대한 연산을 연속적으로 배치
- 캐시 효율성 향상
- 제약 조건 검증 간소화

### 3. 패딩

고정된 테이블 크기를 위해 빈 행을 패딩으로 채웁니다:

```
RwRow {
rw_counter: 0,
is_write: false,
tag: RwTableTag::Start,
// ... 모든 필드 0
}
```

## 제약 조건

### 1. Lookup 제약

EVM Circuit에서 RW Table로의 lookup:

```
meta.lookup_any("rw lookup", |meta| {
vec![
q_enable,
rw_counter,
is_write,
tag,
id,
address,
field_tag,
storage_key,
value,
value_prev,
aux1,
aux2,
]
});
```

## 2. State Circuit 제약

- **정렬 검증**: Lexicographic ordering 확인
- **상태 일관성**: 읽기 값이 이전 쓰기 값과 일치
- **MPT 증명**: 상태 변경에 대한 Merkle 증명

# 디버깅 팁

### RW Table 덤프

```
// RW 연산 출력
for (i, rw) in  rws.iter().enumerate() {
println!("{}: {:?}", i, rw);
}
```

### 일반적인 오류

1. **rw_counter 불일치**: EVM Circuit과 State Circuit의 rw_counter가 맞지 않음
2. **정렬 오류**: RW 연산이 올바르게 정렬되지 않음
3. **값 불일치**: 읽기 값이 이전 쓰기 값과 다름

# 관련 문서

- State Circuit - RW Table을 사용하는 State Circuit
- EVM Circuit - RW Table을 조회하는 EVM Circuit
- Bus-Mapping - RW 연산 생성 과정
- Lookup 메커니즘 - Lookup 동작 원리

# 참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 617)
- **Witness**: `zkevm-circuits/src/witness/rw.rs`

- **Bus-Mapping**: `bus-mapping/src/operation.rs`